

Гайд для новичков

[Ссылка на оригинал этого доку](#)

1. Что читать?

Истинно идеального нет ничего. C'est la vie.

Идеально для начинающих:

- [М. Уэйт, С. Прата, Д. Мартин – Язык программирования С.](#) (небольшая и клевая, но устаревший стиль примеров).
- [С. Прата, "Язык Си, лекции и упражнения \(6 изд.\)](#) (без VPN) – расширенный и более современный вариант предыдущей книги (но сильно больше по объему).

Для что-то знающих – параллельно еще книга Кернигана и Ритчи “Язык С” (но там устаревший стиль кода!):

- [Б. Керниган и Д. Ритчи. Язык С.](#)

Более развернутый список литературы по системному программированию: [тык](#) (без VPN) и немного по С/С++: [тык](#).

2. Какую среду программирования использовать?

В общем, любую. Но ту, с которой вы чувствуете себя уверенно.

Для начинающих и сомневающихся лучше всего подойдет эта сборка среды программирования Code::Blocks (**под Windows**):

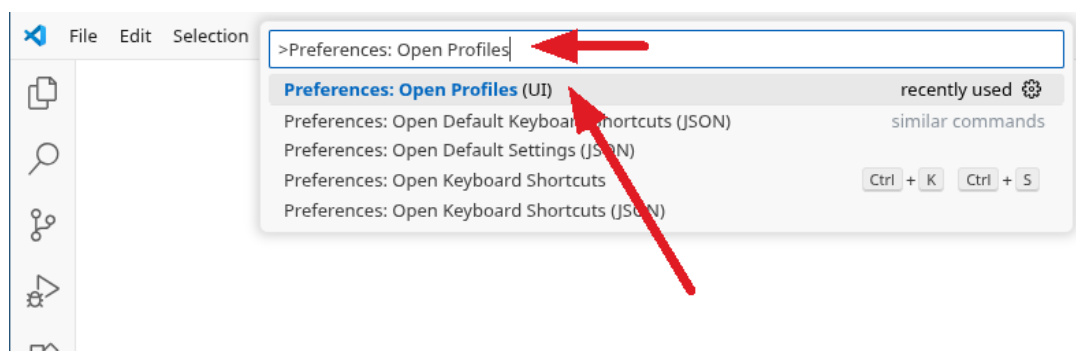
[Скачать среду программирования Code::Blocks \(версия с правильными настройками, она же сборка коdbлокса от Дедá\): http://storage.ded32.net.ru/Lib/TX/codeblocks-mingw-setup-rar.exe](#)

Что делать после установки? **Ctrl+Shift+N** —> создается новый файл —> пишем простейшую прогу —> далее **F9** или **F5** для компиляции (перевода в машинный код) и запуска проги.

Для Linux и MacOS — среда программирования **VSCode**. Установка **рекомендуемых настроек**:

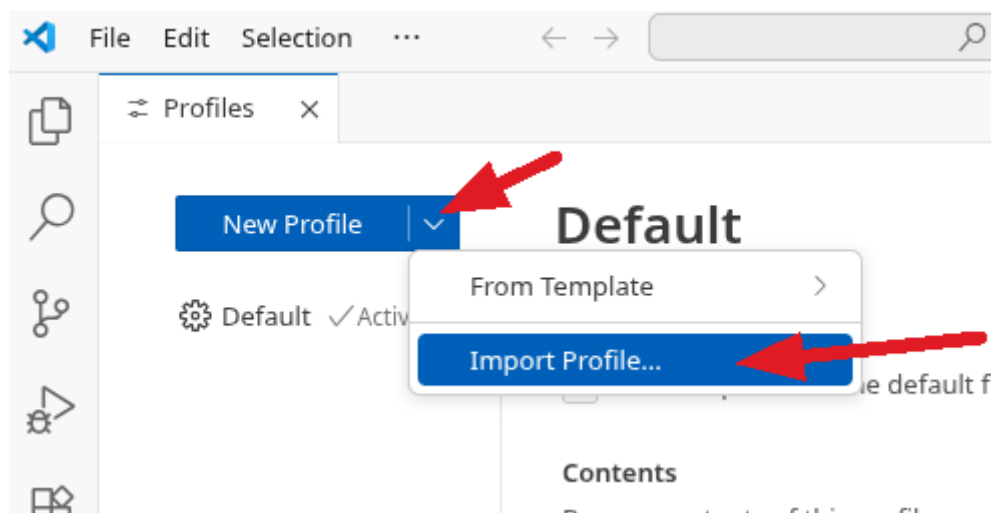
1. Заходим в VSCode.
2. Нажимаем комбинацию **Ctrl+Shift+P** (Windows и Linux) или **Shift+Command+P** (MacOS). (Эта комбинация открывает command palette) и пишем:

Preferences: Open profiles (UI)

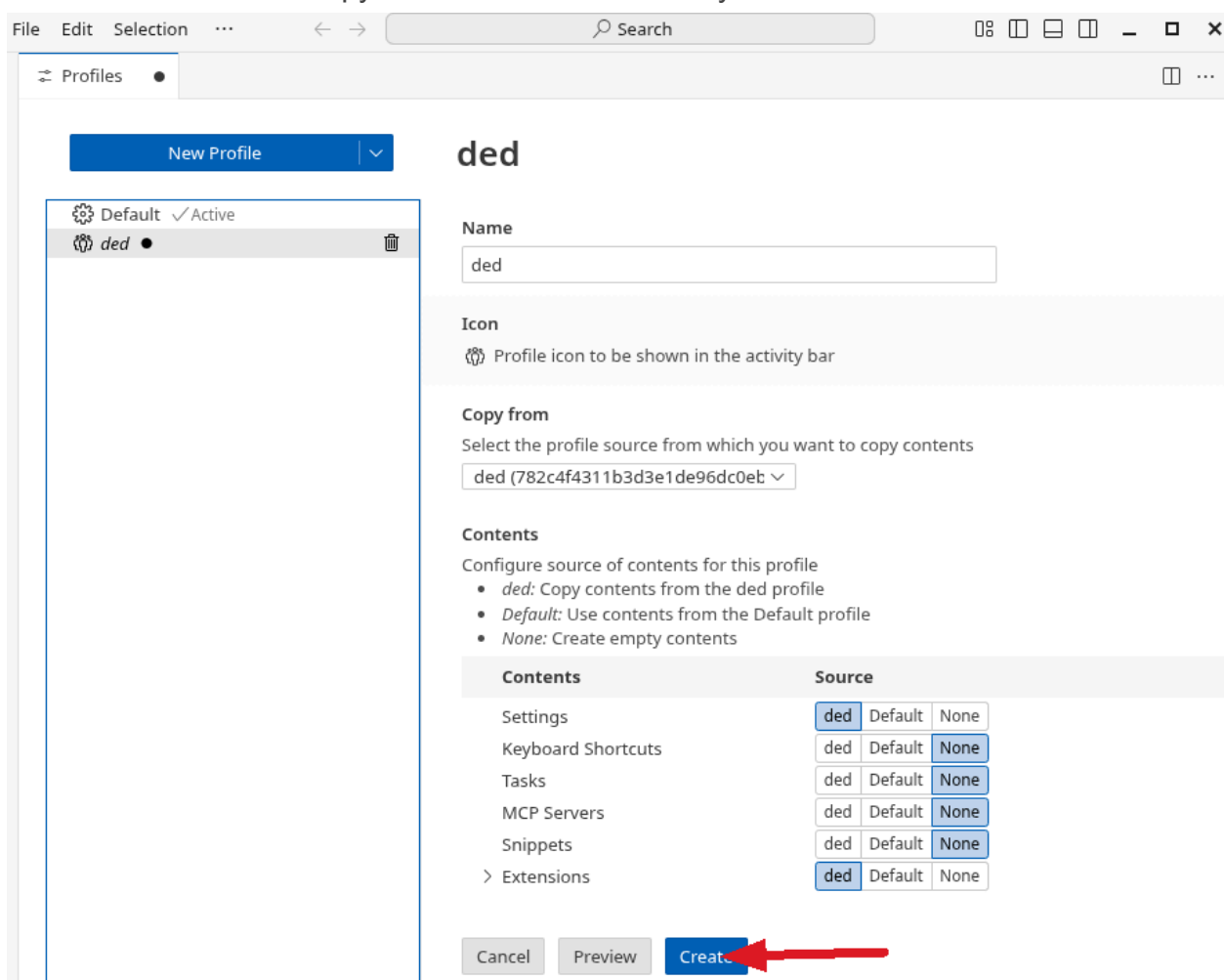


Откроется меню настройки профилей.

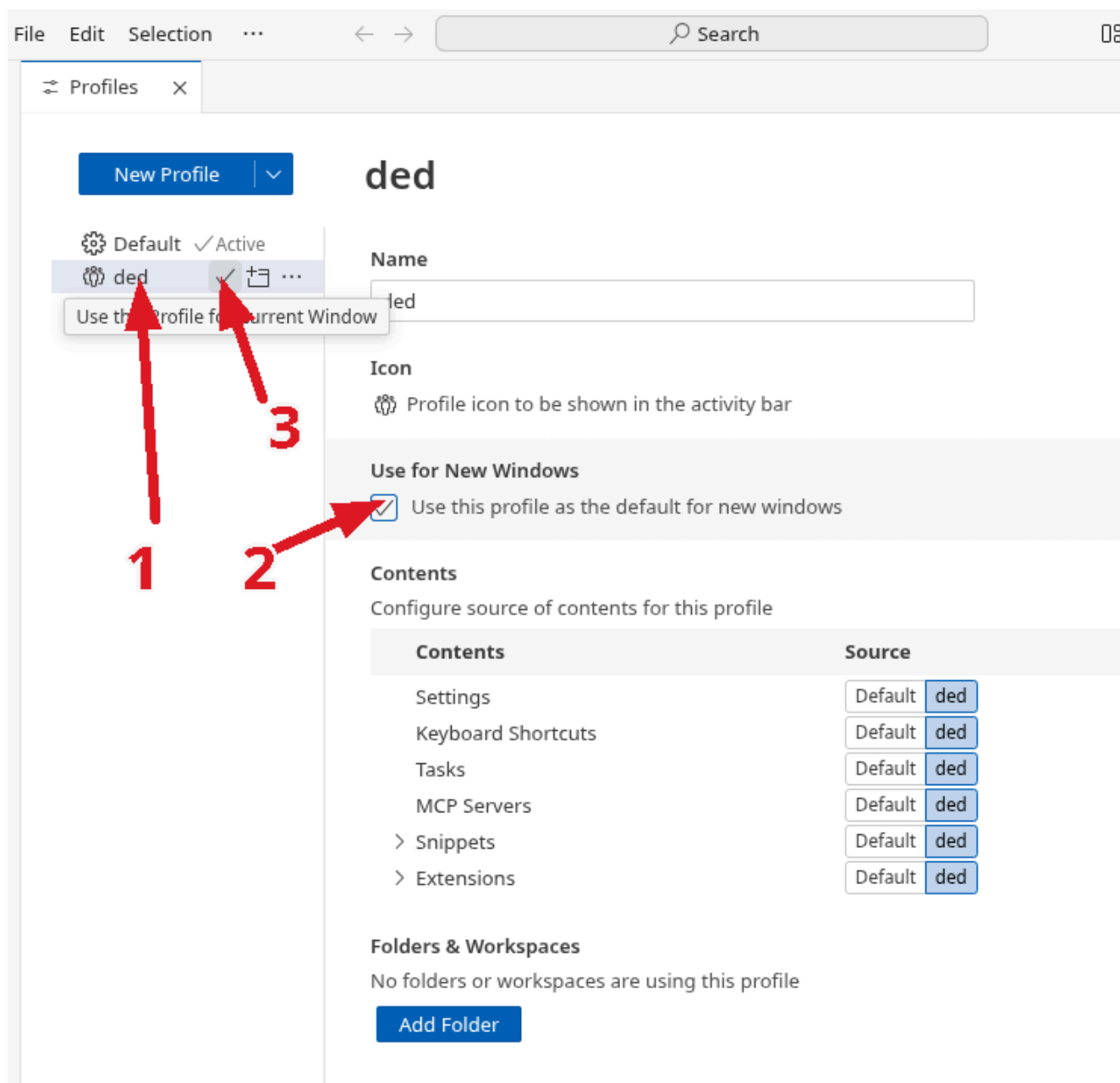
3. Нажимаем на значок “v” у кнопки “New profile”. Выбираем “Import profile”. В всплывшем окне вставляем ссылку: <https://gist.github.com/mishaglik/782c4f4311b3d3e1de96dc0eb825905b>



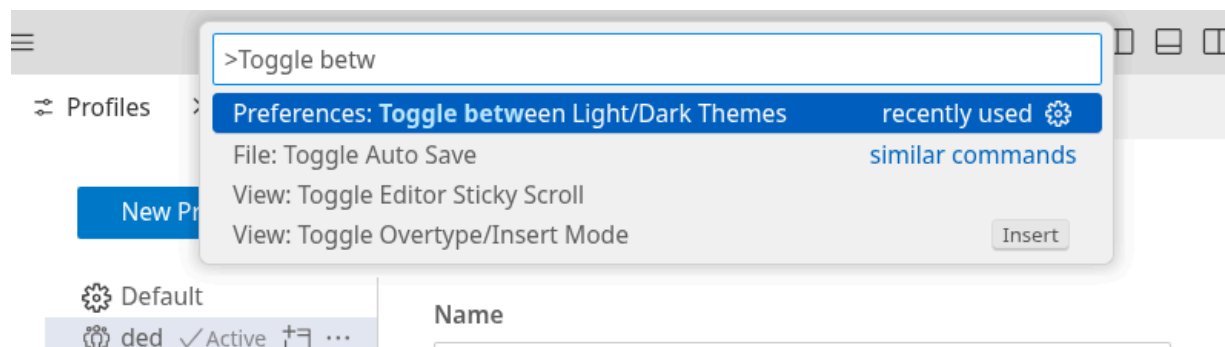
4. После того как тема загрузилась нажимаем кнопку “Create”.



5. Затем выбираем “Use for new windows” и около нового профиля выбираем галочку, чтобы активировать для текущего окна.



6. Если хотите себе светлую тему, то снова прожимаем **Ctrl+Shift+P** и пишем “Toggle between Light/Dark themes”.



7. Для правильного вывода русских символов в консоль в ОС Windows нужно поменять кодировку файлов по умолчанию. Нажимаем комбинацию **Ctrl+,** (это вход в настройки) и пишем в строке поиска **encoding**. Ставим галочку **Files: Auto Guess Encoding**, для параметра **Files: Encoding** выбираем **Cyrillic (Windows 1251)**.
8. Теперь настройки соответствуют стандарту 0xDED и любой ментор с удовольствием будет разглядывать ваш код :)

Заметьте, что чем навороченнее среда, тем продвинутое надо быть, чтобы не запутаться и все настроить нормально. Вы хотите быть крутыми, поэтому, конечно, сразу захотите все самое сложное. Но этого делать не стоит: вы потеряете слишком много времени, которого уже и так осталось мало до летней школы. Лучше углубиться в чтение книжек (см. пункт 1) и написание простейших программ в простой среде программирования. Поэтому, не рвитесь использовать Visual Studio (это не VS Code), CLion или любые другие среды программирования, предназначенные для мега-больших проектов, когда есть более простые и понятные. Вы потеряете очень много времени на ее настройку и понимание, вместо того, чтобы активно кодить в это время. После того, как научитесь программировать на хорошем уровне, сможете выбрать среду программирования по душе.

3. Компиляция из консоли, терминала, командной строки (в основном для тех, кто в Linux или MacOS)

Компиляция – это перевод файла в машинный код, без этого ее нельзя запустить. В это время программа также проверяется на типичные ошибки.

Если у вас VSCode, откройте в ней консоль (терминал).

Либо открываем консоль в операционной системе. В Windows рекомендуется установить Far Manager, в Linux – Midnight Commander (mc) или far Manager for Linux (far2l). В MacOS консоль называется “Терминал”.

Если вы в Windows и у вас **не** сборка коdbлокса Деда (см. п. 2 выше в тексте), вам придется самим разбираться с установкой и запуском компилятора.

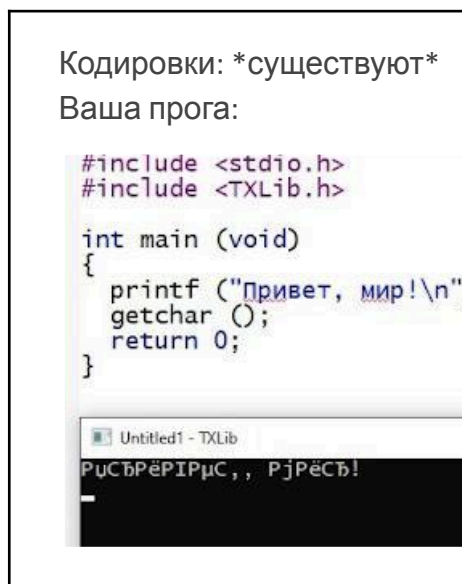
Переходим в рабочую папку (где находятся тексты ваших программ, например, в файле FILENAME.CPP) и выполняем:

```
g++ FILENAME.cpp *флаги* (см. пункт 10: флаги)
```

FILENAME.cpp - это имя файла с текстом проги. Выходной файл будет называться a.exe (a.out для Linux).

4. Русский язык и Windows

К сожалению, русский язык в консоли Windows – это проблема, так как для него есть разные кодировки символов. Погуглите, что это (да, это входит в школьную программу по информатике – минутка душнильства). ~~Так что если дружите с английским, юзайте его или потратьте еще больше времени переходя на Linux.~~ Итак, что же делать, чтобы вы могли лицезреть нужный вам вывод текста, а не вот такое вот:



Если у вас CodeBlocks от Деда (см. выше), то надо добавить в начало файла **ПЕРВОЙ**¹ строкой:

```
#include <TXLib.h> // И все, и все, больше ниче не надо, траляляля!!!2
```

Если у вас другая среда программирования или вы не устанавливали CodeBlocks от Деда, то решение точно такое же, но надо установить TXLib отсюда:
http://sourceforge.net/project/platformdownload.php?group_id=213688.

P.S. Существует конечно же более грамотные методы решения этой задачи, но о них вы узнаете позже. Больно умный, Мишаня!) Чем тебе тхлиб не угодил???) Целая библиотека одним гигантским файлом... ага ага, а если файлов будет два, то они один из них потеряют, проверено Это понятно. Но ради пары строк, фикса кодировки добавляем графическую либу... а настройки консольного окна программно ты тоже будешь рассказывать?... чтобы шрифт там был уникодовый с русскими буквами Дам ссылку на каникулы)))) Вот когда разберутся, как в тхлибе это устроено (там функция txSetLocale) то пусть сами и пишут, собственно, так это и задумано И ЕЩЕ ТАМ МОЖНО РИСОВАТЬ КОТИКОВ!!! На котиков я ничего не могу противопоставить. Мяу!³ А ЭТО СВЯЩЕННО!!!!!! АААААА!!!! НЕ ТРОЖЬ КОТИКОВ!!!! Я УПРЛС давай оставим здесь этот диалог, не будем стирать, а?))) Неплохо. Но надо как то выделить. Курсивом ща бахну

¹ Главное, что **до** подключения стандартных библиотек.

² Дед упрлс. Это бывает.

³ Глас свыше, среагировавший на упоминание о котиках⁴.

⁴ Конечно же, Котиках.

А вот скрин проги с работающим русским языком:



Как видите, все работает.

5. Немного про C и C++

Мы пишем на C, но используем компилятор g++, предназначенный для C++, так как он лучше по ряду причин.

Если у вас есть опыт работы на C++, то, вкратце: не используйте ничего из библиотеки STL (все что требует `std::`, и не пишите `using namespace std`⁴), методов классов, ссылок (указатели – можно). Вместо `cin` и `cout` используйте функции `scanf` и `printf`.

Также: Предупреждения во время компиляции (warnings) – на самом деле это **скрытые ошибки**. То есть любое предупреждение компилятора должно трактоваться как ошибка. Так что если вы не можете пофиксить warning, то спросите, как это сделать правильно, а не забивайте на него.

6. Linux?

Linux – это название семейства операционных систем.

Если уже работали с Linux, то okay. Но если никогда не сталкивались, не стоит разбираться сейчас, это займет очень много времени и вы пролетите. Бывает, настройка Linux занимает недели, а тем временем другие вместо этого прогают, а вы от них невосполнимо отстанете.

Если вы на Windows, то пишите на Windows. В течение первого семестра не будет ничего, специфичного для Linux. Через месяц-другой можно поставить WSL, это подсистема, работающая в Windows, но эмулирующая сервисы Linux. Также можно поставить и настроить виртуальную машину, это уже практически полноценный Linux, только в настройке она будет удобнее. После первого семестра на каникулах уже поставите Linux окончательно.

⁴ За это мы прям убиваем. Натурально, да.

7. Первое задание

- **Написать программу, решающую квадратное уравнение в действительных числах.**

Несмотря на то, что задание выглядит очень простым, к нему будет много требований с точки зрения профессионального программирования. Об этом будет на лекциях, плюс будут рассказывать менторы, когда вы им покажете первый работающий вариант. Приготовьтесь к тому, что переделывать (рефакторить) и/или что-то добавлять вы будете несколько раз. Это абсолютно нормально в профессиональном программировании. Наша цель – написать не просто задачу, а самое лучшее квадратное уравнение в вашей жизни (включая бесконечное количество ее последующих реинкарнаций).

До летней школы было бы неплохо добраться до простейшего варианта этого задания, будет затравочный вариант решения (прототип). На первом занятии мы разберем, как его надо улучшить с профессиональной точки зрения.

8. На летней школе:

8.1. Что делать, когда написал первый вариант?

Начинайте работу с ментором. Ваш ментор найдет вас на летней школе. Если нет - спросите любого ментора, он поможет вам найти вашего.

8.2. Что делать дальше?

Продолжайте работу с ментором. Мы, менторы, здесь, чтобы помочь вам. (Дед – тоже ментор, если лекцию не читает.) Не бойтесь задавать вопросы, даже те, которые кажутся глупыми. На самом деле глупых вопросов не бывает! Молчание – худшая стратегия. Активный диалог – лучшая стратегия. Спрашивайте, спрашивайте и еще раз спрашивайте.

Работа над заданиями – итеративный процесс. Следуйте наставлениям вашего ментора и просите его посмотреть код снова.

Следующие задания будут интереснее и объемнее, их условия вам выдаст ваш ментор, как только вы сделаете предыдущее задание.

9. Котики и Коллективный Разум

- Отправленный котик на фото, особенно свежепокормленный⁵ и свежепоглаженный, особенно на видео, на порядок увеличивает вероятность успеха любого действия! Если у вас есть какие-либо вопросы по проге – не стесняйтесь, задавайте. Мы, менторы (админы этой беседы), будем помогать вам разбираться. Чат “Коллективный разум” создан специально для этих целей. Пишите вопросы именно туда, а не в личные сообщения, чтобы:

- 1) получить более комплексное мнение от разных менторов,
- 2) этот вопрос отпал у всех остальных, кто стесняется его спросить.

⁵ Полторашку только не кормите. Она уже старая, у нее диета, ее кормят специальным ветеринарным кормом.



Рис. 2. Не без кота.

10. Флаги компиляции

Рекомендованные флаги для компиляции:

Windows:

```
-Wshadow -Winit-self -Wredundant-decls -Wcast-align -Wundef -Wfloat-equal -Winline -Wunreachable-code
-Wmissing-declarations -Wmissing-include-dirs -Wswitch-enum -Wswitch-default -Weffc++ -Wmain -Wextra -Wall -g -pipe
-fexceptions -Wcast-qual -Wconversion -Wctor-dtor-privacy -Wempty-body -Wformat-security -Wformat=2 -Wignored-qualifiers
-Wlogical-op -Wno-missing-field-initializers -Wnon-virtual-dtor -Woverloaded-virtual -Wpointer-arith -Wsign-promo
-Wstack-usage=8192 -Wstrict-aliasing -Wstrict-null-sentinel -Wtype-limits -Wwrite-strings -Werror=vla -D_DEBUG
-D_EJUDGE_CLIENT_SIDE
```

Linux:

```
-D_DEBUG -ggdb3 -std=c++17 -Wall -Wextra -Weffc++ -Waggressive-loop-optimizations -Wc++14-compat -Wmissing-declarations
-Wcast-align -Wcast-qual -Wchar-subscripts -Wconditionally-supported -Wconversion -Wctor-dtor-privacy -Wempty-body
-Wfloat-equal -Wformat-nonliteral -Wformat-security -Wformat-signedness -Wformat=2 -Winline -Wlogical-op -Wnon-virtual-dtor
-Wopenmp-simd -Woverloaded-virtual -Wpacked -Wpointer-arith -Winit-self -Wredundant-decls -Wshadow -Wsign-conversion
-Wsign-promo -Wstrict-null-sentinel -Wstrict-overflow=2 -Wsuggest-attribute=noreturn -Wsuggest-final-methods
-Wsuggest-final-types -Wsuggest-override -Wswitch-default -Wswitch-enum -Wsync-nand -Wundef -Wunreachable-code
-Wunused -Wuseless-cast -Wvariadic-macros -Wno-literal-suffix -Wno-missing-field-initializers -Wno-narrowing
-Wno-old-style-cast -Wno-varargs -Wstack-protector -fcheck-new -fsized-deallocation -fstack-protector -fstrict-overflow
-flto-odr-type-merging -fno-omit-frame-pointer -pie -fPIE -Werror=vla
-fsanitize=address,alignment,bool,bounds,enum,float-cast-overflow,float-divide-by-zero,integer-divide-by-zero,leak,nonnull-attribute,null,object-size,return,returns-nonnull-attribute,shift,signed-integer-overflow,undefined,unreachable,vla-bound,vptr
```

MacOS:

```
-D_DEBUG -ggdb3 -std=c++17 -Wall -Wextra -Weffc++ -Wc++14-compat -Wmissing-declarations -Wcast-align -Wcast-qual
-Wchar-subscripts -Wconversion -Wctor-dtor-privacy -Wempty-body -Wfloat-equal -Wformat-nonliteral -Wformat-security
-Wformat-signedness -Wformat=2 -Winline -Wnon-virtual-dtor -Woverloaded-virtual -Wpacked -Wpointer-arith -Winit-self
-Wredundant-decls -Wshadow -Wsign-conversion -Wsign-promo -Wstrict-overflow=2 -Wsuggest-override -Wswitch-default
-Wswitch-enum -Wundef -Wunreachable-code -Wunused -Wvariadic-macros -Wno-missing-field-initializers -Wno-narrowing
-Wno-old-style-cast -Wno-varargs -Wstack-protector -fcheck-new -fsized-deallocation -fstack-protector -fstrict-overflow
-fno-omit-frame-pointer -Wlarger-than=8192 -fPIE -Werror=vla
-fsanitize=address,alignment,bool,bounds,enum,float-cast-overflow,float-divide-by-zero,integer-divide-by-zero,nonnull-attribute,null,return,returns-nonnull-attribute,shift,signed-integer-overflow,undefined,unreachable,vla-bound,vptr
```

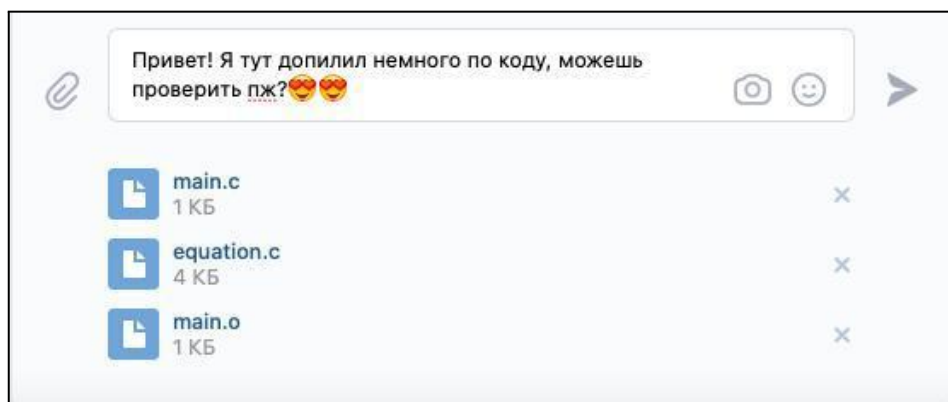

В сборке CodeBlocks от Деда они встроены по умолчанию. Более полную инфу о них см. в конце этого дока: <http://storage.ded32.net.ru/Lib/TX/TXUpdate/Doc/HTML.ru>.

11. Github

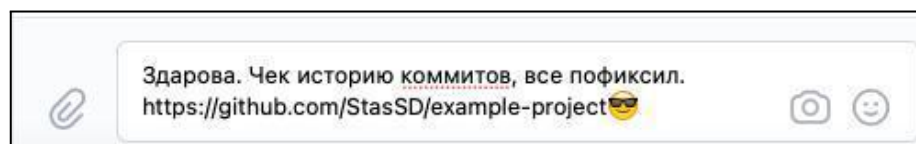
11.1. Введение

Для начальных сведений достаточно знать, что гитхаб ([GitHub](#), не путать с просто git, подробнее будет ниже, не бойтесь) – это облачное хранилище, на которое разработчики загружают свои проекты, работают вместе над ними удаленно, просматривают код друг друга, ~~перекидываются в комментариях картинками с котиками, улюлюкают над багами~~ и делают прочую прогерскую работу. В рамках всего курса вы будете постоянно пользоваться им для того, чтобы показывать код своим менторам.

Многие из вас с огнем в глазах и сердце будут делать что-то такое со своими менторами:



Так делать не надо, так как у менторов лапки и шалящие нервишки :) Итак, намного лучше, когда вы делаете что-то вроде этого:



А вот что в истории коммитов видно:

Закрыл матан 👤 mishaglik committed now
Убрал очепятку в документах 👤 mishaglik committed 15 seconds ago
Исправил обработку ввода 👤 mishaglik committed 37 seconds ago
Поправил кодстайл. 👤 mishaglik committed 1 minute ago

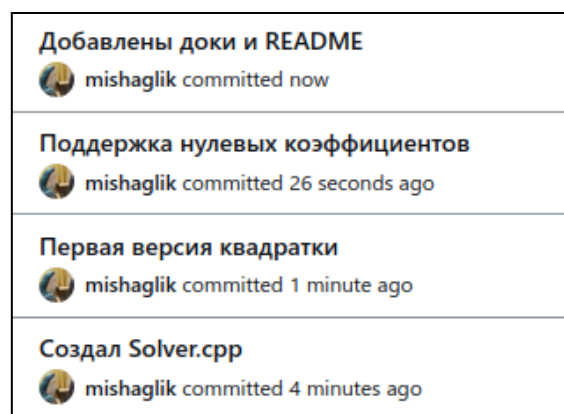
11.2. Что за кошмар? Какие ещё коммиты, гиты, гитхабы?

При редактировании файлов (работе в фотошопе, написании кода и пр.) часто возникает необходимость в сохранении версий файла. К примеру, чтобы иметь возможность откатить

изменения назад, когда Ctrl+Z уже недоступно (например, закрыта вкладка с файлом). Решение такой задачи “в лоб”, как правило, представляет собой создание кучи файлов или папок с номерами и названиями вида “Project.c”, “Project(1).c”, “Project_updated.c”, “Project_final.c”, “Project_final_final.c” и т.д. Очевидно, что это как громоздко, так и неудобно, вызывает путаницу. По этой причине были придуманы **системы контроля версий**, наиболее популярной среди которых является git.



В рамках системы контроля версий в пределах одного **репозитория** (по большому счёту, проекта; к примеру, одной задачи у Деда) поддерживается история и последовательность версий. **Коммиты** в таком случае выполняют роль пунктов (шагов, “снимков”, состояний) в этой истории. Каждому коммиту присущи: номер (на самом деле это хеш), сообщение (краткое описание изменений, к примеру, “добавлена документация функций”, “поддерживаются нулевые коэффициенты”, “исправлены ошибки сравнения чисел double” и пр.) и список изменений. Примером последовательности коммитов может быть такая история:



Github – это сайт, он хранит репозитории, что позволяет разработчику не переживать по поводу потери данных на компе от проблем с операционной системой и всякого иного треша.

Ладно, мы не на лекции по матану, так что время перейти к практике и, наконец, разобраться как работать с git’ом и GitHub’ом.

11.3. Создание профиля

Для создания своего репозитория на гитхабе:

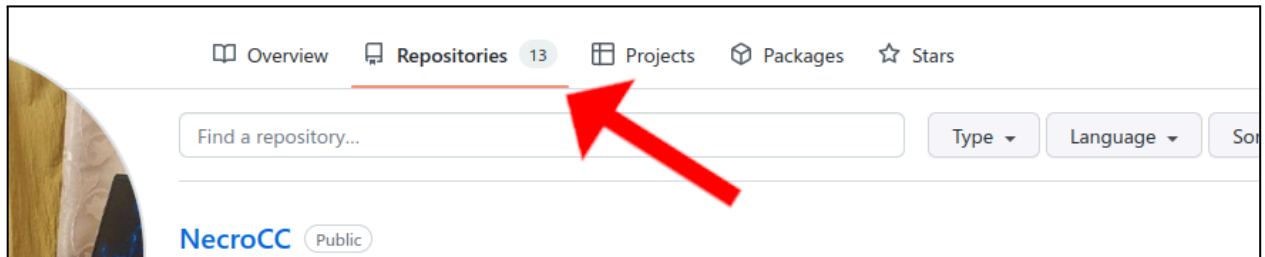
1. Переходим на [GitHub](https://github.com).
2. Регистрируемся.
3. Радуемся первым успешным шагам, так как теперь вы успешный юзер гитхаба!

11.4. Создание первого репозитория

Репозиторий (репо) – это грубо говоря папка с вашим крутым проектом, просто эта директория находится в облачном хранилище. Тут все просто, структура не отличается от обычной папки на вашем компьютере.

Для создания репозитория:

1. Заходим в своем профиле во вкладку repositories



2. Жмем на эту большую зеленую кнопку **New**
3. Вводим имя проекта. Например SquareSolver (решатель квадратных уравнений)
4. Не обращаем пока внимания на кнопки по типу создания README.md и .gitignore
5. Делаем его публичным, чтобы можно было хвастаться вашим перфомансом друзьям и менторам
6. Нажимаем на кнопку создания:

A screenshot of the 'Create a new repository' form on GitHub. The form has several sections: 'Owner' and 'Repository name' (with 'SquareSolver' entered and a green checkmark), 'Description (optional)' (with 'Simple square equation solver.' entered), 'Public/Private' selection (with 'Public' selected), 'Initialize this repository with:' section (with 'Add a README file' and 'Add .gitignore' options), and 'Choose a license' section (with 'License: None' selected). A red arrow labeled '1' points to the 'Repository name' field. A red arrow labeled '2' points to the 'Public' radio button. A red arrow labeled '3' points to the 'Create repository' button at the bottom. The button is green and says 'Create repository'.

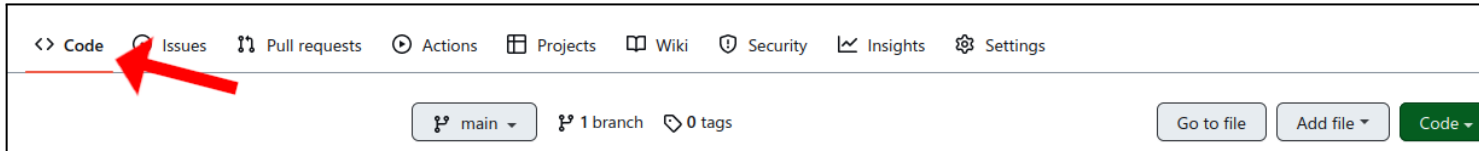
7. Теперь, когда вас просят показать код с вашего репозитория, достаточно просто зайти в него и...
8. ...скопировать url ссылку сверху из окошка адреса в браузере.
9. Гитхаб вас поздравит с первым репозиторием! Это круто.

11.5. Заливаем хороший код файлики для ревью

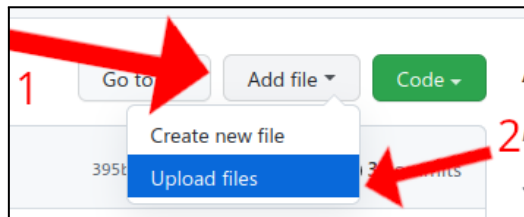
Пустой репозиторий не интересный, надо бы залить туда файлики с кодом с вашего компа. Мы сейчас пойдем самым мега простым путем, не супер правильным. Потом расскажем, как это делать лучше.

Для этого:

1. Перейдите в ваш репозиторий в раздел code:

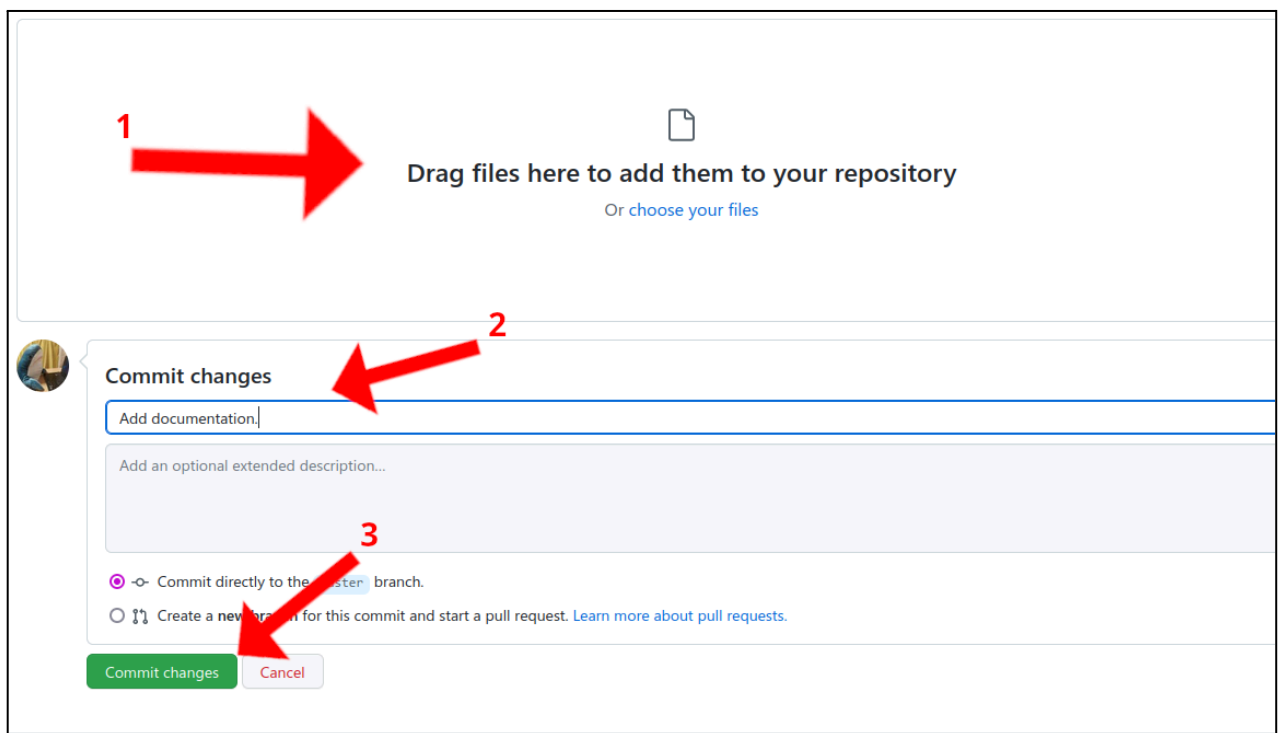


2. Нажмите на Add file , а после чего Upload file:



Ну дальше выбираем файлы с компьютера и загружаем их.

3. Придумываем хорошее описание для коммита. Например “Add documentation”. Или “Fixed input issues”. Или “Я пофиксил баги ввода!” Или “Я блин забыл вообще написать ввод чисел, как оно до этого работало???”



11.6. А можно удобнее?

Если вам неудобно постоянно заливать файлы через браузер (а это обязательно должно быть вам неудобно!), можно воспользоваться одним из способов, позволяющих загрузить все файлы проекта сразу буквально в 3 клика.

Способ 1: GitHub Desktop

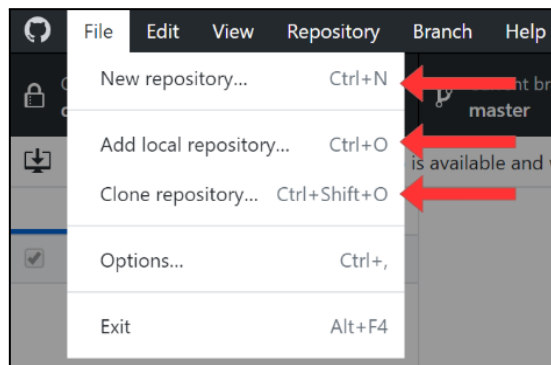
Этот способ для вас, если вы используете CodeBlocks. Github Desktop – это специальное приложение для работы с Github.

Скачать его можно по ссылкам:

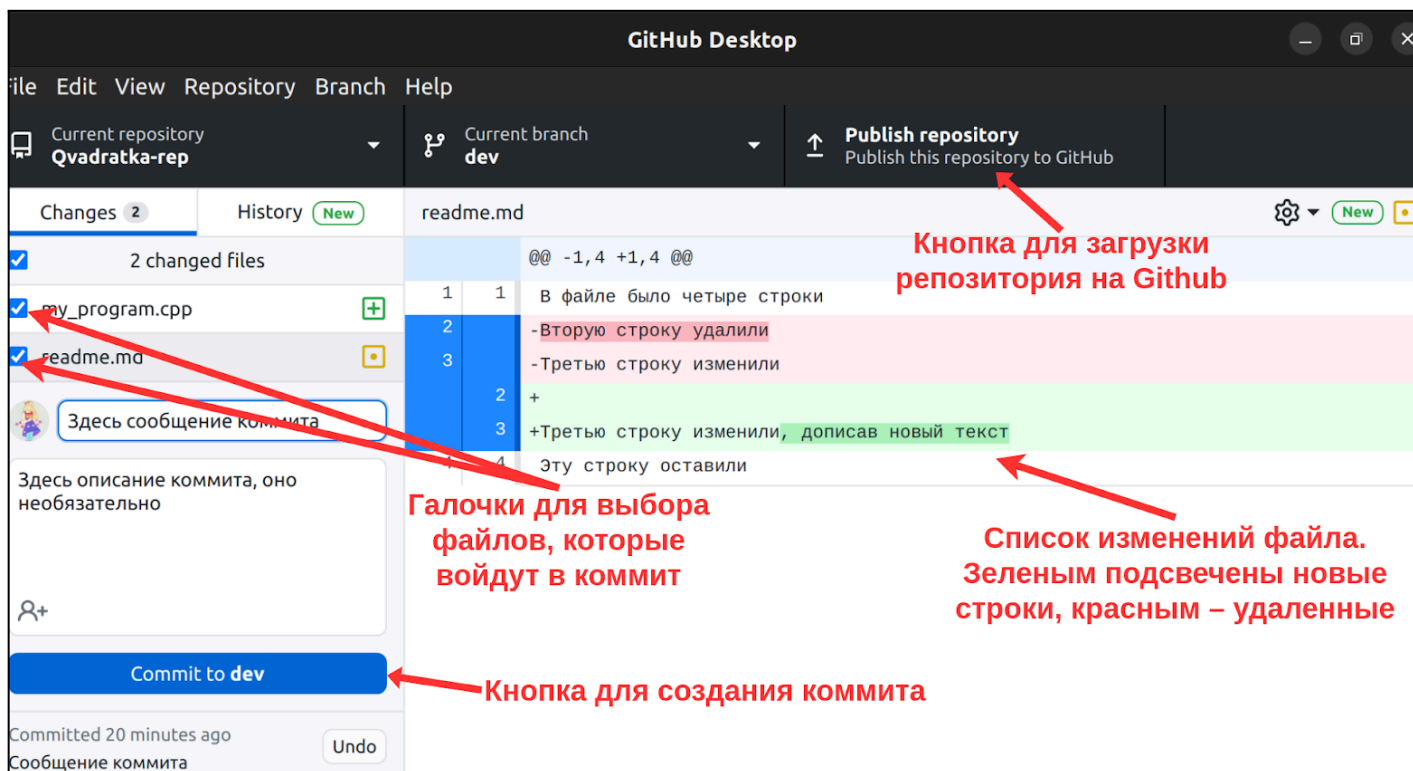
- Для Windows/Mac: <https://desktop.github.com>
- Для Linux: <https://github.com/shiftkey/desktop/releases>

После установки вам нужно будет авторизоваться в приложении через ваш аккаунт на Github.

Чтобы добавить репозиторий, откройте раздел “File” из верхнего меню. Там вы можете создать новый репозиторий, клонировать (скачать к себе на компьютер) репозиторий из github или открыть уже существующий локальный репозиторий.

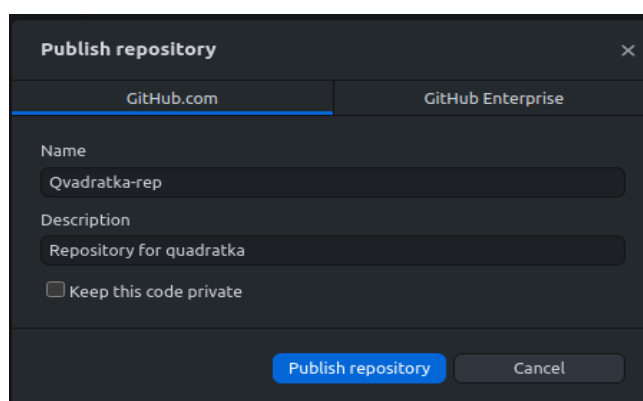


После создания репозитория, вы можете приступить к написанию кода. Как только вы решили сохранить очередную версию своей задачи (сделать коммит), откройте GitHub Desktop, откройте ранее созданный репозиторий. Измененные файлы появятся в панели слева. Вы можете поставить галочки на тех файлах, которые войдут в коммит (да, не обязательно фиксировать изменения во всех файлах разом, можно добавить в коммит только некоторые файлы, но поначалу это вам вряд ли пригодится). В правой части окна будет отображаться, какие изменения вы внесли в файл с момента последнего коммита. В нижней части левого меню будет окошко, чтобы ввести краткое описание изменений (summary) и, по желанию, подробное описание изменений (description).



После того, как вы поставили галочки на файлах, которые войдут в коммит и ввели сообщение коммита, можно нажимать кнопку “Commit to ...”. Вместо троеточия будет название ветки, у меня это dev, у вас, скорее всего, будет master или main. Поздравляю, вы сделали коммит! Осталось только залить этот коммит на Github (эта операция называется push).

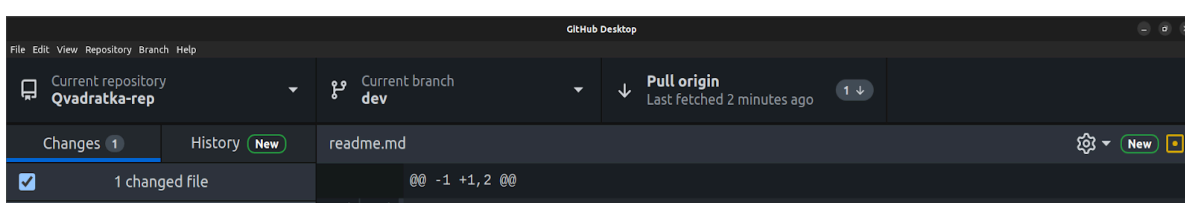
Для того, чтобы загрузить локальный репозиторий на гитхаб, надо нажать кнопку “Publish repository” в верхнем меню. В открывшемся окошке введите имя репозитория (обязательно), и описание репозитория (по желанию). Обязательно снимите галочку “Keep this code private”, иначе ваш репозиторий будет доступен только вам, и менторы не смогут посмотреть ваш код. Настоящие тру пацаны никогда не делают приватные репо.



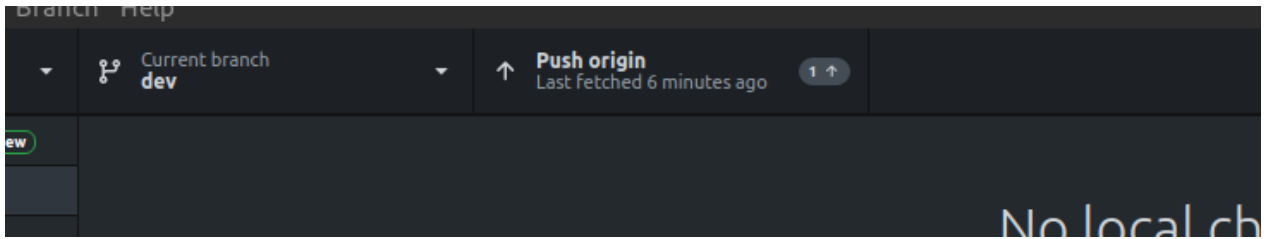
Как только вы ввели имя репозитория и сняли галочку “Keep this code private”, можно нажимать “Publish repository”. Поздравляем, ваш репозиторий теперь на гитхабе!

Чтобы залить последующие изменения на гитхаб, нужно:

1. Внести изменения в файлы. Гитхаб Десктоп сразу заметит это:



2. Ввести сообщение коммита и сделать коммит (то есть сохранить все ваши изменения).
3. Нажать на кнопку “Push origin”, после создания коммита она появится там, где раньше была кнопка “Publish repository”, – в верхнем меню.



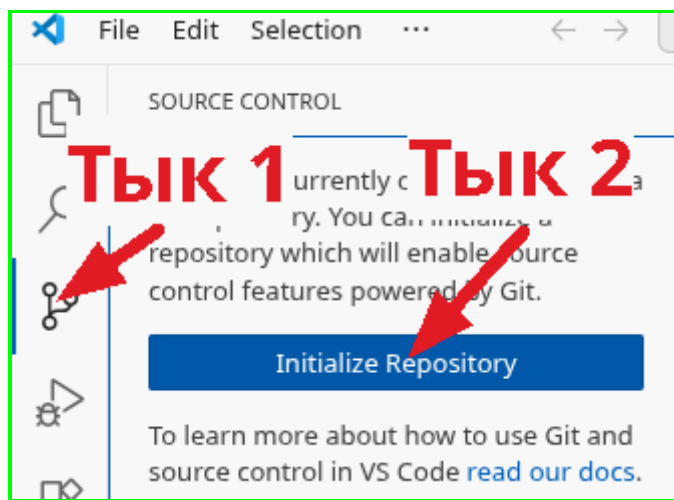
После этого измененные файлы автоматически сохраняются в облако, все изменения появятся в репозитории на Гитхабе и менторы смогут увидеть их. Теперь, если вы даже выбросите свой комп (зачем он вам?), ваши файлы навечно останутся на Гитхабе. Voila.

Способ 2: Git прямо в VSCode

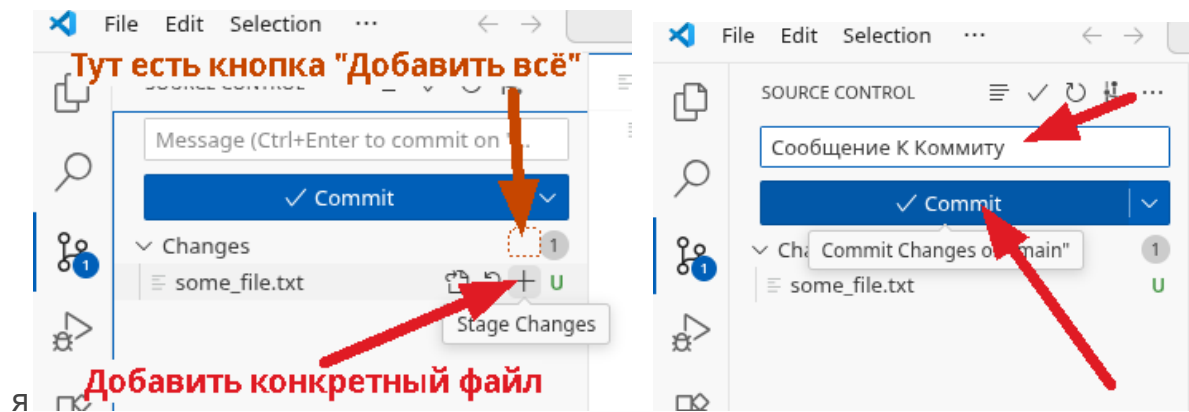
В VSCode поддержка git уже есть по умолчанию. Однако придётся поставить сам git (скачиваем с официального сайта): <https://git-scm.com/downloads>.

После установки откройте папку со своим проектом в Visual Studio Code через меню Файл -> Открыть папку (File->Open Folder).

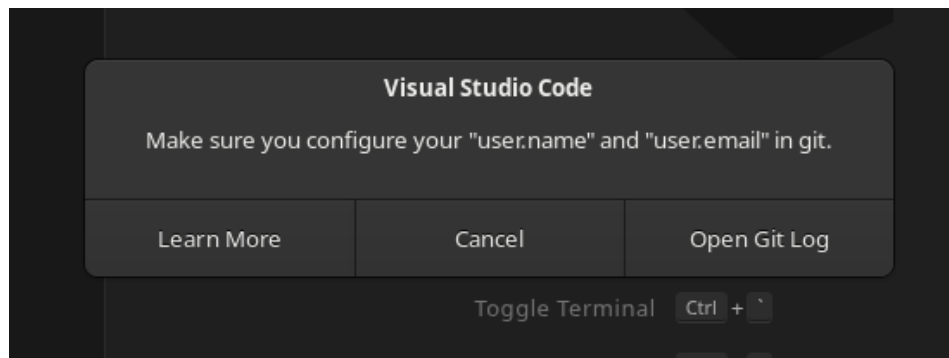
В левой панели выберите вкладку Source Control:



Нажмите кнопку "Initialize repository". Это создаст git-репозиторий в этой папке. После этого уже можно создавать коммиты. Сначала требуется добавить файл с изменениями (кнопка '+' рядом с изменённым файлом, либо добавить всё кнопкой '+' в строке 'Changes'), потом ввести описание коммита (Commit message) и нажать синюю кнопку 'Commit':

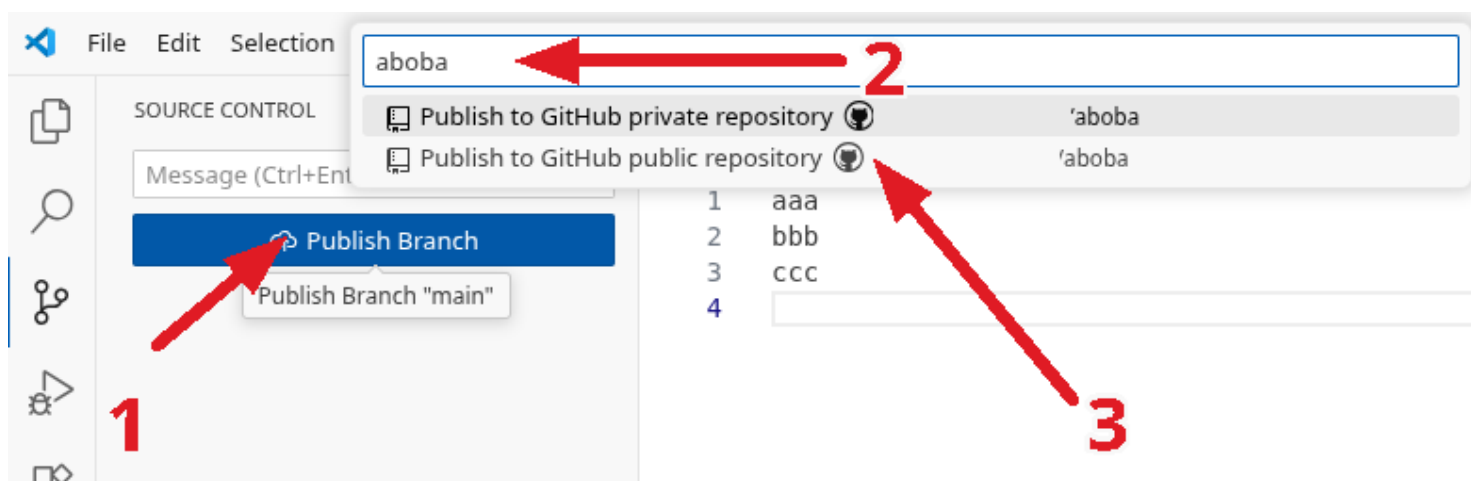


В первый раз может вылезти вот такое сообщение об ошибке:

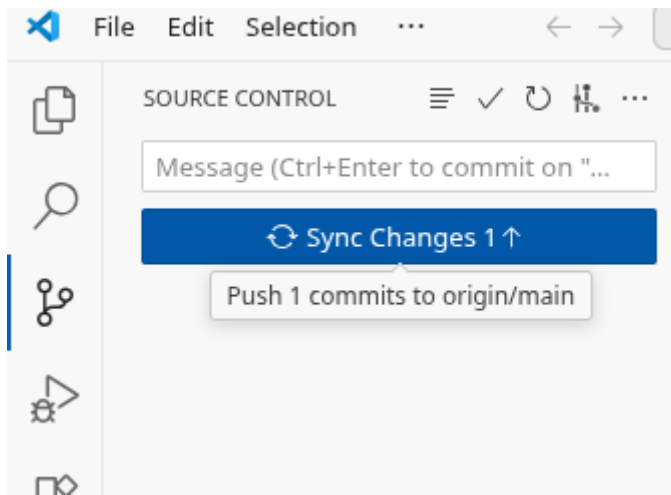


Это фиксируется командами `git config set user.name Ваш_ник_на_github` и `git config set user.email Ваша_почта` (указанная при регистрации на гитхабе) в консоли VSCode.

После этого надо нажать "Publish branch". Авторизуемся... И вот, ваш репозиторий уже сохранен в облако на Гитхабе.



После публикации кнопка "Publish branch" заменится на "push" (именно так называют обновление существующего репозитория в облаке).



ВНИМАНИЕ!

Независимо от способа работы с Гитхабом, важно размещать каждый проект в отдельной папке. Не надо класть исходники, например, квадратки, в ту же папку, где уже лежат ваши примеры типа `hello_world.c` и школьные проекты на питоне. Во-первых, вы сами скоро запутаетесь в этом, когда квадратка начнёт разрастаться, во-вторых ментору в этом будет разобраться ещё сложнее, чем вам, и в-третьих, папки и репозитории жестко связаны: одна папка – один репозиторий.

Репозиторий стоит создавать прямо в той же папке, где вы пишете код.

Git diff.

Нет, Великий Git Diff!

Теперь давайте поговорим о правой части экрана GitHub Desktop, в которой отображаются изменения, которые вы внесли (в VSCode аналогичный интерфейс можно открыть кликом по файлу в списке изменений). Это очень полезная область, с ее помощью можно отловить простые ошибки и опечатки. Давайте разберем на примере.

Итак, вы работаете над большим проектом. Вы занятой человек и у вас одновременно на компе запущена и среда программирования, браузер, Telegram, ~~и еще~~ и много чего. И вот переключаясь между окнами, туда сюда набирая разный текст, вы нечаянно стёрли один символ в коде... Вроде бы. И не заметили. Хм. Ну может тогда компилятор скажет, что я ошибся? Нет, вам не повезло. Компилятор молчит.

А на следующий день программа работает некорректно. Хм странно. Только вчера этот же код работал... Как найти, что изменилось. Можно конечно пристально посмотреть...

```

#include "MGK/Logger.h"
#include "MGK/Utils.h"
#include "Backend.h"
#include "Elfer.h"
#include "StFrame.h"
#include <stdarg.h>
#include <string.h>

const size_t INIT_SZ = 32;

extern const size_t INT_SZ = 8; // Size of target integer type.

#define ASM(format, ...) {ByteBufferAppendf(&context->asmBuf, format "\n\t", ## __VA_ARGS__);
#define UPDATE_ADDR(of) *(int*)&context->binBuf.buf[of - 4] = (int)(context->binBuf.size - of)
#define LABEL_REGISTER(lbl) ASM("Lnu:", lbl); LabelRegister(&context->labelBuf, lbl, context->binBuf,

const size_t regStackSz = 14;
const size_t stdcallRegsSz = 6;

const Reg stdcallRegs[stdcallRegsSz] = {Reg::RDI, Reg::RSI, Reg::RDX, Reg::RCX, Reg::R8, Reg::R9};
const Reg regStack[regStackSz] = {Reg::RAX, Reg::RDI, Reg::RSI, Reg::RDX, Reg::RCX, Reg::R8, Reg::R9, Reg::R10, Reg::R11, Reg::R12, Reg::R13, Reg::R14, Reg::R15};

void backend(const Node* root, const char* filename){
    LOG_ASSERT(root != NULL);
    LOG_ASSERT(filename != NULL);

    LOG_FILLER("BACKEND START");

    BackendContext context = {};
    newByteBuffer(&context.asmBuf);
    newByteBuffer(&context.binBuf);
    newByteBuffer(&context.labelBuf.buf);
    newByteBuffer(&context.funcLabelsBuf);

    ByteBufferAppendf(&context.asmBuf,
        ".intel_syntax noprefix\n"
        ".global _start\n"
        "_start:\n");

    LabelReserve(&context.labelBuf, 1); // Take label 0;
    createFrame(&context, NULL, root);
    codeGen(&context, root);
    closeFrame(&context);
    xExit(&context);

    deleteNF(context.nf);

    char* asmFilename = (char*)calloc(strlen(filename) + 5, sizeof(char));
    strcpy(asmFilename, filename);
    strcat(asmFilename, ".s");
    FILE* asmFile = fopen(asmFilename, "w");
    free(asmFilename);
    LOG_ASSERT(asmFile != NULL);
    fwrite(context.asmBuf.buf, sizeof(char), context.asmBuf.size, asmFile);

    genElf(filename, context.binBuf.buf, context.binBuf.size);

    deleteByteBuffer(&context.asmBuf);
    deleteByteBuffer(&context.binBuf);
    deleteByteBuffer(&context.labelBuf.buf);
    deleteByteBuffer(&context.funcLabelsBuf);
}

int getNFuncArgs(const Node* node){
    if(node == NULL)
        return 0;
    if(node->type == NodeType::IDENTIFIER){
        LOG_WARNING("Found var [%p] %d", node, node->data.id);
        return 1;
    }
    return getNFuncArgs(node->left) + getNFuncArgs(node->right);
}

int evaluateArguments(BackendContext* context, const Node* node){
    LOG_ASSERT(context != NULL);
    if(node == NULL) return 0;
    if(node->type == NodeType::OPERATOR && node->data.opr == Operator::COMMA){
        return
            evaluateArguments(context, node->right) +
            evaluateArguments(context, node->left);
    }
    else{
        codeGen(context, node);
        XPUSH(regStack[--context->regStackUsed]);
        return 1;
    }
}

int main(int argc, char* argv[]){
    LOG_ASSERT(argc > 2);
    Tree* tree = readTree(argv[1]);

    backend(tree->root, argv[2]);

    nodeListDtor(tree->list);
    free(tree);
}

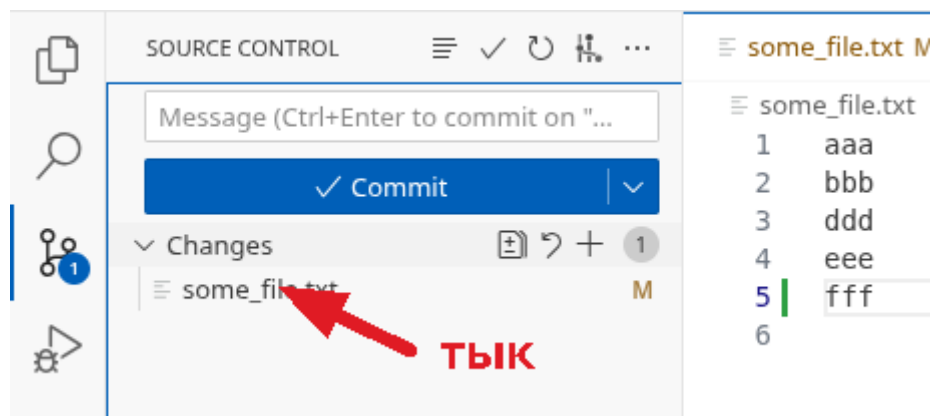
void createFrame(BackendContext* context, Nameframe* par, const Node* node){
    LOG_ASSERT(context != NULL);
    XPUSH(Reg::RBP);
    context->nf = createNF(par);

    int n = getVarCnt(node) * INT_SZ;
    if(node->type == NodeType::OPERATOR && node->data.opr == Operator::FUNC){
        n = getVarCnt(node->right->right) * INT_SZ;
        n += MIN(6, getNFuncArgs(node->right->left)) * INT_SZ;
    }
    XMOVRR(Reg::RBP, Reg::RSP);

    if(par){
        for(size_t i = 1; i < context->regStackUsed; ++i){
            xPushR(context, regStack[i]);
        }
    }
}

```

М-да. Слишком много... и это довольно сложный код, далеко не первая задача. И как же найти что со вчера поменялось? Но вы как разумный человек конечно же сделали коммит на моменте когда у вас код работал. (Если нет, то Ф. Нужно больше коммитов! Делайте коммит на каждой стабильной позиции!!!) На помощь придет git diff. (его можно получить просто посмотрев в окно github desktop или (в vscode) кликнув на файл из списка изменений во вкладке Source Control)



Он показывает отличия вашей текущей версии кода от последнего коммита. (Или между двумя коммитами).

```
563
564 int evaluateArguments(BackendContext* context, const Node* node){
565     LOG_ASSERT(context != NULL);
-   if(node == NULL) return 0;
566+   if(node = NULL) return 0;
567     if(node->type == NodeType::OPERATOR && node->data.opr == Operator::COMMA){
568         return
569             evaluateArguments(context, node->right) +
570             evaluateArguments(context, node->left);
571     }
572     else{
573         codeGen(context, node);
574         XPUSH(regStack[--context->regStackUsed]);
575         return 1;
576     }
577 }
```

Красное - то, что было.

Зеленое - то, что стало.

Опа! Да вот же она, опечатка на 566-й строке.

Из примера выше можно сделать вывод – читайте диффы (от англ diff – различие, так называется область, в которой отображаются изменения), они помогут вам отловить ошибки и опечатки, которые может пропустить даже компилятор.

Здесь нужна картинка “собачка в огне Fine”. Вот она:



In case of fire

- 1. git commit
- 2. git push
- 3. leave building

Справа - то, что народная мудрость говорит о git во время пожара. Соблюдайте правила пожарно-охранной сигнализации! Вы еще услышите ее в общаге в 6 утра. Конечно же, ее включают, чтобы вы не забыли сделать коммит!

Как не заливать в облако ненужные вещи, или про файл .gitignore

Когда вы начнёте писать код, вы, возможно, заметите, что кнопка “*добавить все изменения*” имеет свойство добавлять всякую гадость, такую как .o и .exe файлы, созданные компилятором. От этого можно избавиться, если создать в репозитории файл с именем “.gitignore”, и в нём просто написать в столбик шаблоны имён нежелательных файлов. Например:

```
*.o  
*.exe  
*.save
```

Всё! Эти файлы больше не залиются на Гитхаб. Нечего им там делать, они все равно создаются заново при каждой компиляции, и их сохранение в облако замедляет перекачку коммита.

Еще более грамотно – использовать Git из консоли или с помощью более продвинутых программ, например, SmartGit или других. Но об этом в кратком гайде мы не будем, а расскажем вам позже.

This Гайд[®] ©[™] подготовили master [Дед](#) и его slaves менторы⁶ (больше менторы, чем [Дед](#)⁷ (они же slaves (но не в том смысле (а в каком том? (а вот в таком (в правильном (норм чуваки а чо (а ниче (ну и че (пойдем выйдем (не пойду, я **K&R**⁸ читаю.))))))))))) VIVA LISP (А это к Винокурову (это норм препод такой (Лисп любит (а почему ЛИСП? а скобочки))))))))))

ОПУБЛИКОВАНО ПОД ВСЕМИ КОПИРАЙТАМИ И КОПИЛЕФТАМИ⁹
ИЗДАТЕЛЬСТВО ФИЗТЕХ КРИНЖ ВОТАУ ПАБЛИШЕРС poltorashka edition
МОСКВА (ДОЛГОПА БЛИН!!!) 2022-2026

⁶ Попросили остаться анонимными, дабы избежать преследования со стороны перовокуров.

⁷ Так-то он отвечает за большую часть кринжа здесь⁸.

⁸ Конечно, имеется в виду K&R⁸.

⁹ Нет дискриминации для правшей и для левшей. А всех несогласных мы погоним взашей⁸.

⁸ Разумеется^{5, 8}.